First-Class and Higher-Order Functions

Learning Outcomes

by the **End** of the **Lecture**, **Students** that **Complete** the **Recommended Exercises** should be **Able** to:

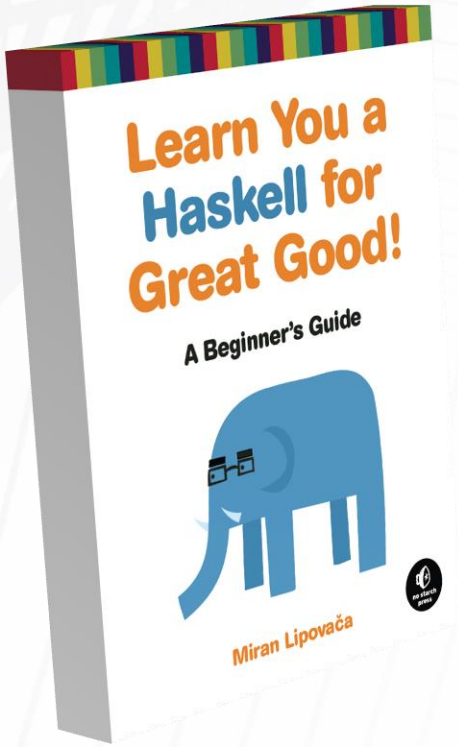
Describe First-Class Objects

Describe Higher-Order Functions

Use the fold, map, and filter Functions

Contrast these with the Corresponding Design Patterns

Reading and Suggested Exercises



Read the following **Chapter(s)**:

Chapter 6:
"Curried functions"
"Some higher-orderism is in order"
"Maps and filters"

First-Class Objects

First-Class Objects (or First-Class Citizens) are programming language entities that can be:

Passed as **Arguments**

Returned by **Procedures**

Stored in a **Variable**

First-Class Objects

in **Most Programming Languages**, **Primitives**
(e.g., integers, floats, etc.) are **First-Class Objects**

in some **Older Languages**, **Linear Collections**
(e.g., arrays, strings, etc.) are **Not First-Class Objects**

n.b., in C you cannot pass an array as an argument and must pass a pointer instead

First-Class Objects

in **Functional Languages**, Functions are
Always First-Class Objects

this **Allows** for **Higher-Order Functions**, that:

Require Functions as **Arguments**
and/or
Produce Functions as **Return Values**

Higher-Order Functions

when **Higher-Order Functions** are **Available**, the **Accumulation**, **Transformation**, and **Selection** (i.e., "**Folding**", "**Mapping**", and "**Filtering**", respectively) **Design Patterns** can be **Revisited**

Higher-Order Functions

there are **Three Fundamental Patterns** for **Recursion** on **Lists** that can be **Widely Applied**

Accumulation (i.e., "**Folding**")

Apply a binary **Operator Between** the **Elements** of a **List**

Transformation (i.e., "**Mapping**")

Call a **Function** on **Every Element** in a **List**

Selection (i.e., "**Filtering**")

Test a **Predicate** for **Every Element** of a **List**

Functions for Transformation

Using the **Recursive Design Patterns**,
How did you **Write Functions** for the following?

Doubling Each Element of a **List**

```
mapDouble [1, 2, 4] = [2, 4, 8]
mapDouble :: [Int] -> [Int]
```

Making a **Horizontal Histogram**

```
mapReplicate [3, 4, 1] = ["***", "****", "*"]
mapReplicate :: [Int] -> [[Char]]
```

Functions for Transformation

Doubling Each Element of a List

```
mapDouble :: [Int] -> [Int]
mapDouble [] = []
mapDouble (h:t) = (2 * h) : mapDouble t
```

Making a Horizontal Histogram

```
mapReplicate :: [Int] -> [[Char]]
mapReplicate [] = []
mapReplicate (h:t) = (replicate h '*') : mapReplicate t
```

Functions for Transformation

it is quite **Obvious** that the **Implementations** are **Very Similar**

in fact it is not unreasonable to think of the **Design Pattern** (i.e., "**Mapping**") as a **Process** that **Accepts** a **Function** as one of its **Inputs** and **Performs** the corresponding **Transformation**

Functions for Transformation

the **Function Argument** that would **Correspond**
to **Doubling Each Element** of a **List** would perform
"**Multiplication by Two**"

the **Function Argument** that would **Correspond**
to **Making a Horizontal Histogram** would perform
"**Replication of an Asterisk**"

What Property do these **Functions** have **In Common**?

Functions for Transformation

both **Multiplication** and **Replication** are **Binary** (i.e., the **Number** of **Arguments** needed is **Two**), but **Multiplying "by Two"** and **Replicating "an Asterisk"** should both be considered **Unary**, since **One** of the **Two Arguments** has **Already** been **Fixed**

Where is the **Other Argument** coming from?
it is the **Next Element** of the **List** to be "**Mapped**"

Functions for Transformation

If there were a **Mapping Function** (`map`) that required (as **Arguments**) **Both** a **Function** and a **List** before performing **Transformation** to **Produce** a **List** as a **Result**, **What** would be the **Type Definition** of this **Mapping Function**?

Functions for Transformation

```
map :: functionArg -> listArg -> listResult
```

With What will you **Replace**

`functionArg`, `listArg`, and `listResult`?

Functions for Transformation

How would you **Write** a **Function** that
Doubles an **Integer**?

now, **Compare**:

the **Previous** `mapDouble` **Function**

`map` called with a "doubling" **Function** as an **Argument**

Functions for Transformation

```
mapDouble :: [Int] -> [Int]
mapDouble [] = []
mapDouble (h:t) = (2 * h) : mapDouble t

double :: Int -> Int
double x = 2 * x

map :: (a -> b) -> [a] -> [b]
```

`mapDouble [1,2,3]` **Vs.** `map double [1,2,3]`

Functions for Transformation

```
*Main> mapDouble [1,2,3]
[2,4,6]
*Main> :t mapDouble
mapDouble :: [Int] -> [Int]
*Main> map double [1,2,3]
[2,4,6]
*Main> :t map
map :: (a -> b) -> [a] -> [b]
*Main> :t map double
map double :: [Int] -> [Int]
```

Functions for Transformation

`map` (from **Prelude**) is a **Higher-Order Function**
with the **Transformation Function** as an **Argument**

the **Transformation Function** is **Applied**
to **Each Element** of the **Second Argument** (i.e., the list)

How could `map` be **Implemented**?

Functions for Transformation

```
mapDouble :: [Int] -> [Int]
mapDouble [] = []
mapDouble (h:t) = (2 * h) : mapDouble t

double :: Int -> Int
double x = 2 * x

map :: (a -> b) -> [a] -> [b]
map f [] = []
map f (h:t) = (f h) : (map f t )
```

Functions for Selection

Using the **Recursive Design Patterns**,
How did you **Write Functions** for the following?

Select Only the **Even Values** from a **List**

```
filterEven [1, 2, 2, 4, 4] = [2, 2, 4, 4]
filterEven :: [Int] -> [Int]
```

Remove all **Instances** of a **Value** from a **List**

```
filterRemove 2 [1, 2, 2, 4, 4] = [1, 4, 4]
filterRemove :: Int -> [Int] -> [Int]
```

Functions for Selection

Select Only the **Even Values** from a **List**

```
filterEven :: [Int] -> [Int]
filterEven [] = []
filterEven (h:t)
   | even h = h : filterEven t
   | otherwise = filterEven t
```

Remove all **Instances** of a **Value** from a **List**

```
filterRemove :: Int -> [Int] -> [Int]
filterRemove x [] = []
filterRemove x (h:t)
   | x == h = filterRemove x t
   | otherwise = h : filterRemove x t
```

Functions for Selection

once again, it is not unreasonable to think of the **Design Pattern** (i.e., "**Filtering**") as a **Process** that **Accepts** a **Predicate** as one of its **Inputs** and **Performs** the corresponding **Selection**

during **Selection**, the **Predicate** is **Tested** using elements from the **List Argument** and **Any Element** that **Satisfies** the **Predicate** is **Selected**

Functions for Selection

the **Predicate Argument** that would **Correspond**
to **Select Only** the **Even Values** from a **List**
would be "**Divides Evenly** by **Two**"

the **Predicate Argument** that would **Correspond**
to **Remove** all **Instances** of a **Value** from a **List**
"**Is Equal** to **Other Argument**"

Do these **Predicates** have the **Same Arity**?

Functions for Selection

If there were a **Filtering Function** (`filter`) that required (as **Arguments**) **Both** a **Predicate** and a **List** before performing **Selection** to **Produce** a **List** as a **Result**, **What** would be the **Type Definition** of this **Filtering Function**?

Functions for Selection

```
filter :: predicateArg -> listArg -> listResult
```

With What will you **Replace**

`predicateArg`, `listArg`, and `listResult`?

Functions for Selection

How would you **Write** a **Function** that
Checks if **Two Values Differ**?

now, **Compare**:

the **Previous** `filterRemove` **Function**

`filter` called with the **Above Function** as an **Argument**

Functions for Selection

```
filterRemove :: Int -> [Int] -> [Int]
filterRemove x [] = []
filterRemove x (h:t)
  | x == h = filterRemove x t
  | otherwise = h : filterRemove x t

isDiff :: Int -> Int -> Bool
isDiff x y = x /= y

filter :: (a -> Bool) -> [a] -> [a]
```

`filterRemove a x` **Vs.** `filter isDiff a x`

Functions for Selection

```
filterRemove :: Int -> [Int] -> [Int]
filterRemove x [] = []
filterRemove x (h:t)
  | x == h = filterRemove x t
  | otherwise = h : filterRemove x t

isDiff :: Int -> Int -> Bool
isDiff x y = x /= y

filter :: (a -> Bool) -> [a] -> [a]
```

`filterRemove a x` **Vs.** `filter (isDiff a) x`

Functions for Selection

`filter` (from **Prelude**) is a **Higher-Order Function** with the **Selection Predicate** as an **Argument**

the **Selection Predicate** is **Tested** against **Each Element** of the **Second Argument** (i.e., the list)

How could `filter` be **Implemented**?

Functions for Selection

```
filterRemove :: Int -> [Int] -> [Int]
filterRemove x [] = []
filterRemove x (h:t)
  | x == h = filterRemove x t
  | otherwise = h : filterRemove x t

isDiff :: Int -> Int -> Bool
isDiff x y = x /= y

filter :: (a -> Bool) -> [a] -> [a]
filter p [] = []
filter p (h:t)
  | (p h)  = h : (filter p t)
  | otherwise = filter p t
```

Functions for Accumulation

Using the **Recursive Design Patterns**,
How did you **Write Functions** for the following?

Compute the **Sum** of the **Elements** of a **List**

```
foldSum [1, 2, 2, 4, 4] = [2, 2, 4, 4]
foldSum :: [Int] -> Int
```

Concatenate all the **Elements** of a **List**

```
foldConcat ['A', 'B', 'C'] = "ABC"
foldConcat :: [[a]] -> [a]
```

Functions for Accumulation

Compute the **Sum** of the **Elements** of a **List**

```
foldSum :: [Int] -> Int
foldSum [] = 0
foldSum (h:t) = h + (foldSum t)
```

Concatenate all the **Elements** of a **List**

```
foldConcat :: [[a]] -> [a]
foldConcat [] = []
foldConcat (h:t) = [h] ++ (foldConcat t)
```

Functions for Accumulation

and again, it is not unreasonable to think of the **Design Pattern** (i.e., "**Folding**") as a **Process** that **Accepts** an **Operator** as one of its **Inputs** and **Performs** the corresponding **Accumulation**

during **Accumulation**, the **Operator** is **Applied Between Every Pair** of **Elements** in the **List Argument**

Functions for Accumulation

the **Operator Argument** that would **Correspond** to **Compute** the **Sum** of the **Elements** of a **List** would be "**+**"

the **Operator Argument** that would **Correspond** to **Concatenate** all the **Elements** of a **List** would be "**++**"

What are the **Operand Types** for these **Operators**?

Functions for Accumulation

If there were a **Folding Function** (`fold`) that required (as **Arguments**) **Both** an **Operator** and a **List** before performing **Accumulation** to **Produce** a **Result**, **What** would be the **Type Definition** of this **Folding Function**?

Functions for Accumulation

```
foldr :: ? -> listArg -> result
```

With What will you **Replace**

`?`, `listArg`, and `result`?

Unfortunately you **Cannot Simply Replace** the **Question Mark** with a **Binary Operator...**

Why Not?

Functions for Accumulation

the **Sum** of the **Elements** of `[1, 2, 3, 4]` is `10`

Consider the **Recursive Sum** of this **List** in **Reverse**

the **Last Step** was **Stop**, because `[] 10` is the **Base Case**

but **Before That...**

Functions for Accumulation

the **Sum** of the **Elements** of `[1, 2, 3, 4]` is `10`

Consider the **Recursive Sum** of this **List** in **Reverse**

the **Last Step** was **Stop**, because `[] 10` is the **Base Case**

Add the **Head** of `[4]` to the **Running Total** `6`

but **Before That...**

Functions for Accumulation

the **Sum** of the **Elements** of `[1, 2, 3, 4]` is `10`

Consider the **Recursive Sum** of this **List** in **Reverse**

the **Last Step** was **Stop**, because `[] 10` is the **Base Case**

Add the **Head** of `[4]` to the **Running Total** `6`

Add the **Head** of `[3, 4]` to the **Running Total** `3`

Add the **Head** of `[2, 3, 4]` to the **Running Total** `1`

Add the **Head** of `[1, 2, 3, 4]` to... **What**?

Functions for Accumulation

a **Higher-Order Function** for **Folding** (as it has been described so far) would **Require** an **Identity Element** to **Act** as the **Initial Value** of the **Running Accumulator** (e.g., Sum, Product, etc.)

the **Identity Element** is **Used** because, by **Definition**, its **Inclusion** has **No Effect** on the **Final Result**

Functions for Accumulation

rather than **Writing** a **Function** for **Addition**,
simply use the **Built-In** "`(+) :: Num a => a -> a - > a`"?

now, **Compare**:

the **Previous** `foldSum` **Function**

`foldr` called with the **Above Function** as an **Argument**

Functions for Accumulation

```
foldSum :: [Int] -> Int
foldSum [] = 0
foldSum (h:t)
  = h + (foldSum t)

foldr :: (a -> b -> b) -> b -> [a] -> b
```

`foldSum x` **Vs.** `foldr (+) 0 x`

Functions for Accumulation

`foldr` (from **Prelude**) is a **Higher-Order Function** with the **Accumulation Operator** as an **Argument**

it **Also Requires** an **Identity Element** as an **Argument**, and the `r` **Indicates** that you are **Folding To** the **Right**

How could `foldr` be **Implemented**?

Functions for Accumulation

```
foldSum :: [Int] -> Int
foldSum [] = 0
foldSum (h:t)
  = h + (foldSum t)

foldr :: (a -> b -> b) -> b -> [a] -> b
foldr op id [] = id
foldr op id (h:t)
  = (op h (foldr op id t))
```

Functions for Accumulation

Not All Binary Operators have a **Identity Value**
(or, at least, **Not** one that is **Useful** in this **Context**)

Minimum Of, for instance, is a **Binary Operator**
for which there is **No Meaningful Identity Element**

Functions for Accumulation

`foldr1` (from **Prelude**) is a **Higher-Order Function** with the **Accumulation Operator** as an **Argument**

it **Does Not Require** an **Identity Element** and, instead, **Uses** the **Final Element** of the **List**

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr1 :: (a -> a -> a) -> [a] -> a
```

Higher-Order Function Example

How do you **Recursively Implement Insertion Sort**?

Which of the **Higher-Order Functions** could you **Use** to **Implement Insertion Sort** (and **How**)?